



VOLUME 12 / RUST BACKEND MASTERY

Fly.io公開 — Docker・CI・運用・復旧

再現可能buildからasset bundle、Docker、Fly設定、migration、smoke、scale、rollback、DRまで公開後を含めて学ぶ

10

深掘り単元

20+

実行・解答例

40+

図・表・画面

実務

設計と障害対応

LEARNING MAP

この巻で身に付けること

デプロイコマンドだけで終わらず、どのartifactがどの設定とdataで動き、失敗時にどう戻すかを具体化します。

到達目標

- ✓ reproducible containerを作れる
- ✓ Topcoat assetとFly port/checkを一致できる
- ✓ 安全なmigrationとsecret rotationを運用できる
- ✓ scale・rollback・DRを検証できる

学習地図

01 release build・reproducibility

本番artifactはCargo.lockを使う--locked、release profile、固定Rust toolchainで作ります。build時刻や環境差を記録し、binary hashとGit commitを対応させます。

02 multi-stage Dockerfile

builder imageでcompileし、runtime imageへbinaryと必要な共有library、assetsだけをコピーします。非root user、明示WORKDIR、small attack surface、.dockerignoreを使います。

03 Topcoat asset release bundle

Topcoat 0.5.0はcompile済みbinary内のasset!参照をCLIが走査します。したがってrelease binaryを先にbuildし、その直後に同じsourceでasset bundleを作ります。

04 fly launch・fly.toml・port

fly launch --no-deployで設定を生成し、アプリがHOST=0.0.0.0、PORT=8080で待ち受け、fly.toml internal_port=8080と一致させます。primary_regionをTursoや利用者に近づけます。

05 secrets・config・rotation

Turso token、session secret、外部API keyはimageやfly.tomlへ書かずfly secretsへ保存します。名前だけを文書化し、値をlogやshell historyへ残さず、rotationを二段階で行います。

06 health check・startup・readiness

Flyのservice checkはtrafficを送れるか判断します。process生存のhealthとDB依存のreadyを分け、startupにmigrationが必要ならgrace periodとtimeoutを現実的に設定します。

07 migration deploy順・release command

schema変更は新旧binary互換を保つexpand/contractで行います。migrationを複数Machineが同時実行しないよう、release commandまたは排他lockを使い、失敗時は新releaseをtrafficへ出しません。

08 deploy・logs・smoke・custom domain

fly deploy後にstatusだけで成功とせず、checks、logs、version、health、auth、CRUD、assetをsmokeします。custom domainではDNSとcertificate状態を確認します。

09 scale・region・cost・Turso latency

Machine数、size、auto stop、concurrency、regionはlatencyとcostを変えます。Rustの低memoryだけを前提にせず、Argon2 CPU、DB接続、idle起動時間を測ります。

10 rollback・backup・disaster recovery・更新

rollbackは前binaryへ戻すだけでなくschema互換、secret、job、assetを含みます。RPO/RTOを決め、Turso復元、Fly image rollback、token revokeを定期演習します。

HOW TO USE THIS VOLUME

読むだけで終わらせない進め方

「予想→実行→観察→一か所変更→回帰テスト」を一単位ごとに繰り返します。写経した行数ではなく、入力・処理・出力・失敗を自分の言葉で説明できるかを進捗にします。

1. 予想

実行前に入力型、出力、失敗箇所を予想します。

2. そのまま実行

最初は変更せず、教材と同じ出力が比較します。

3. 一か所変更

値または型を一つだけ変え、差分の理由を説明します。

4. 回帰テスト

直した失敗を自動テストへ残し、再発を防ぎます。

STEP 1

--locked

STEP 2

release profile

STEP 3

toolchain pin

STEP 4

artifact hash

毎回そろえる確認コマンド

● ● ● 単元を終える前の品質確認

```
cargo fmt --check
cargo clippy --all-targets -- -D warnings
cargo test --locked --all-targets
```

対象	この教材の基準	混在を防ぐ理由
Rust	2024 Edition / rust-version 1.95以上	editionとMSRVを明示し、環境差を再現可能にする
Topcoat	crate / CLI ともに 0.5.0固定	early-stageのため、mainブランチの別構文を混ぜない
DB・認証	Turso/libSQL、Argon2id、Topcoat Session	一つのRustプロセスで責務とデータの流れを追える
公開	Docker + Fly.io、HOST=0.0.0.0、PORT=8080	ローカルとコンテナの待受差を明示する

重要な版の前提 Topcoatは公式がearly-stage / experimentalと明記しています。本教材はcrateとCLIを **0.5.0** に固定し、mainブランチの別構文を混ぜません。

UNIT 01

release build・reproducibility

UNIT 01・CONCEPT

release build・reproducibility

本番artifactはCargo.lockを使う--locked、release profile、固定Rust toolchainで作ります。build時刻や環境差を記録し、binary hashとGit commitを対応させます。

この単元の到達点

- --locked
- release profile
- toolchain pin
- artifact hash

なぜバックエンドで必要か

deploy後にどのsourceと依存から作られたbinaryか追跡でき、同じcommitから再buildして比較できます。



先に答え

source commit、lock、toolchain、artifact digestを一組で記録します。

確認する問い

release build・reproducibilityを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 01・MENTAL MODEL

release build・reproducibilityを図でつかむ



本番artifactはCargo.lockを使う--locked、release profile、固定Rust toolchainで作ります。build時刻や環境差を記録し、binary hashとGit commitを対応させます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	--locked	--locked — 入力と前提を確認し、境界を曖昧にしない。
2	release profile	release profile — 値がどの順で変換されるかを追跡する。
3	toolchain pin	toolchain pin — 失敗を再現し、型またはログから原因を特定する。
4	artifact hash	artifact hash — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

release build・reproducibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 01・MINIMUM CODE

まず動く最小コード

release build・reproducibilityだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
rustc --version
cargo test --locked --all-features
cargo build --locked --release
Get-FileHash .\target\release\rustnote.exe -Algorithm SHA256
```

● ● ● 実行コマンド

```
cargo run
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 01・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>rustc --version</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>cargo test --locked --all-features</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>cargo build --locked --release</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>Get-FileHash .¥target¥release¥rustnote.exe - Algorithm SHA256</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が`--locked`の条件を満たすか確認する。
- ✓ `release profile`を実行し、途中の型と状態を追う。
- ✓ `toolchain pin`で失敗を成功経路から分離する。
- ✓ `artifact hash`により、出力と副作用を検証する。

型を声に出す `release build`・`reproducibility`に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 01 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
Finished release
SHA256 <digest> rustnote.exe
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 01 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
#[route(GET "/version")]async fn version()->Result<String>{Ok(format!("version={} commit={}",env!("CARGO_PKG_VERSION"),option_env!("GIT_SHA").unwrap_or("unknown"))))}
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

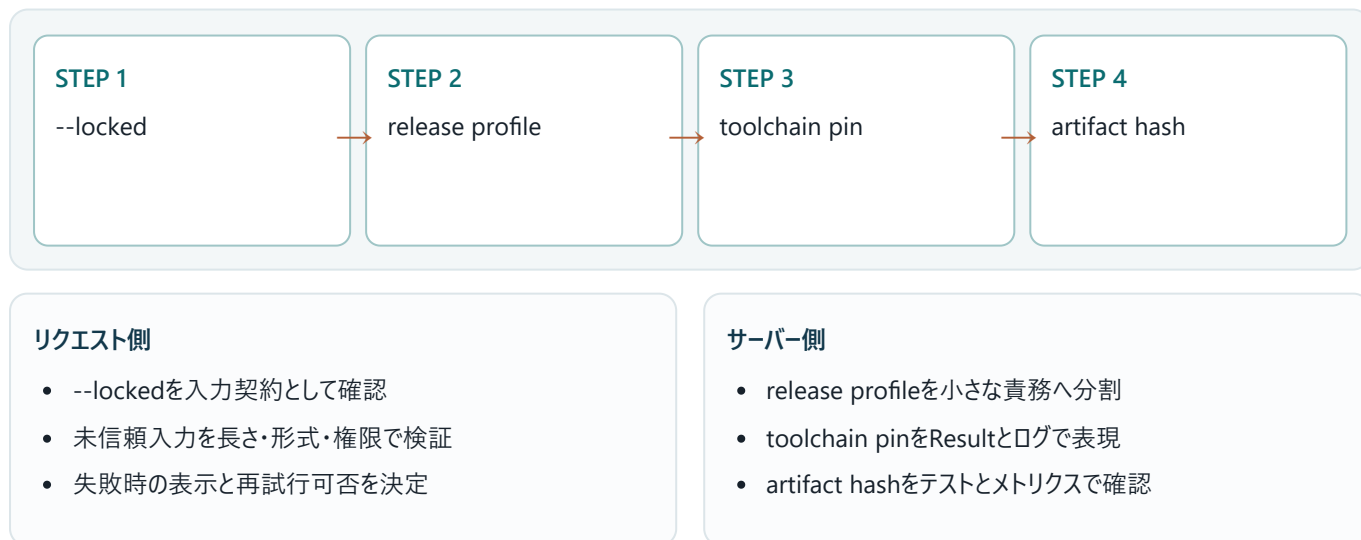
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 01 · BACKEND APPLICATION

バックエンドのどこで使うか

deploy後にどのsourceと依存から作られたbinaryが追跡でき、同じcommitから再buildして比較できます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。

- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 01・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	--lockedに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	release profileの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	toolchain pinまたはartifact hashの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 01・DEBUGGING

release build・reproducibilityを再現して切り分ける

release build・reproducibilityの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: --locked
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: release profile
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: toolchain pin
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: artifact hash

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	--locked	--locked — 入力と前提を確認し、境界を曖昧にしない。
2	release profile	release profile — 値がどの順で変換されるかを追跡する。
3	toolchain pin	toolchain pin — 失敗を再現し、型またはログから原因を特定する。
4	artifact hash	artifact hash — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 source commit、lock、toolchain、artifact digestを一組で記録します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 01 · PRACTICE

手を動かす演習

version endpoint/package versionと短いcommit hashを埋め、秘密なしで表示してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 01・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
#[route(GET "/version")]async fn version()->Result<String>{Ok(format!("version={} commit={}",env!("CARGO_PKG_VERSION"),option_env!("GIT_SHA").unwrap_or("unknown")))}
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ source commit、lock、toolchain、artifact digestを一組で記録します。
- ✓ release build・reproducibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 02

multi-stage Dockerfile

UNIT 02 · CONCEPT

multi-stage Dockerfile

builder imageでcompileし、runtime imageへbinaryと必要な共有library、assetsだけをコピーします。非root user、明示WORKDIR、small attack surface、.dockerignoreを使います。

この単元の到達点

- builder stage
- runtime stage
- non-root
- minimal context

なぜバックエンドで必要か

0.5.0のbundle出力target/assetsをbinary隣の/app/assetsへコピーします。build stageのsourceやCargo registryをruntimeへ入れません。



先に答え

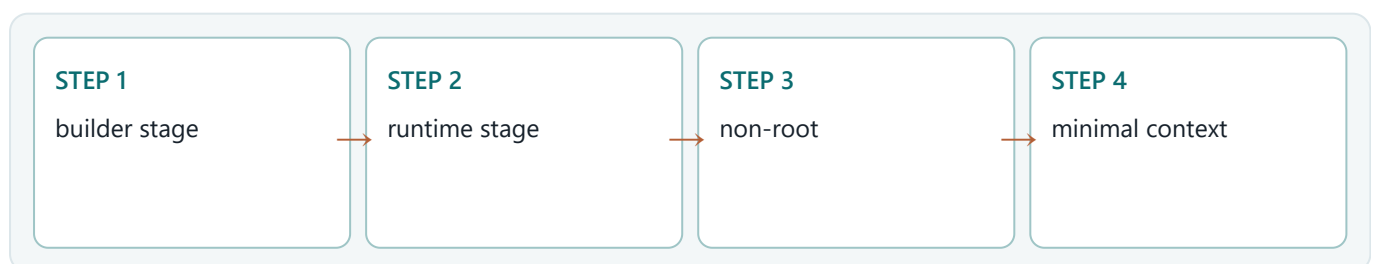
runtime imageへbinary・assets・必要libraryだけを非rootで配置します。

確認する問い

multi-stage Dockerfileを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 02 · MENTAL MODEL

multi-stage Dockerfileを図でつかむ



builder imageでcompileし、runtime imageへbinaryと必要な共有library、assetsだけをコピーします。非root user、明示WORKDIR、small attack surface、.dockerignoreを使います。図では左から右へ処理を追います。各箱の入口では入力の種類と信頼度、出口では結果

の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	builder stage	builder stage — 入力と前提を確認し、境界を曖昧にしない。
2	runtime stage	runtime stage — 値がどの順で変換されるかを追跡する。
3	non-root	non-root — 失敗を再現し、型またはログから原因を特定する。
4	minimal context	minimal context — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

multi-stage Dockerfileとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 02 · MINIMUM CODE

まず動く最小コード

multi-stage Dockerfileだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
FROM rust:1.97-bookworm AS build
WORKDIR /app
COPY Cargo.toml Cargo.lock ./
COPY src ./src
COPY assets ./assets
RUN cargo install topcoat-cli --version 0.5.0 --locked
RUN cargo build --locked --release && topcoat asset bundle --release
FROM debian:bookworm-slim
RUN useradd --system --uid 10001 app
WORKDIR /app
COPY --from=build /app/target/release/rustnote /app/rustnote
COPY --from=build /app/target/assets /app/assets
USER app
CMD ["/app/rustnote"]
```

●●● 実行コマンド

```
docker build --progress=plain -t rustnote:local .
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 02 ・ LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
FROM rust:1.97-bookworm AS build	この行が受け取る値、作る値、副作用の有無を確認します。
WORKDIR /app	この行が受け取る値、作る値、副作用の有無を確認します。
COPY Cargo.toml Cargo.lock ./	この行が受け取る値、作る値、副作用の有無を確認します。
COPY src ./src	この行が受け取る値、作る値、副作用の有無を確認します。
COPY assets ./assets	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力builder stageの条件を満たすか確認する。
- ✓ runtime stageを実行し、途中の型と状態を追う。
- ✓ non-rootで失敗を成功経路から分離する。
- ✓ minimal contextにより、出力と副作用を検証する。

型を声に出す multi-stage Dockerfileに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 02 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
docker image contains /app/rustnote and /app/assets
process uid=10001
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 02 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
docker build -t rustnote:local .
docker run --rm --entrypoint id rustnote:local
docker run --rm --entrypoint ls rustnote:local -la /app
docker run --rm -p 8080:8080 -e HOST=0.0.0.0 -e PORT=8080 rustnote:local
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 02 · BACKEND APPLICATION

バックエンドのどこで使うか

0.5.0のbundle出力target/assetsをbinary隣の/app/assetsへコピーします。build stageのsourceやCargo registryをruntimeへ入れません。



リクエスト側

- builder stageを入力契約として確認
- 未信頼入力を長さ・形式・権限で検証
- 失敗時の表示と再試行可否を決定

サーバー側

- runtime stageを小さな責務へ分割
- non-rootをResultとログで表現
- minimal contextをテストとメトリクスで確認

境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 02 ・ FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	builder stageに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違ふ	runtime stageの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	non-rootまたはminimal contextの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。

✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 02・DEBUGGING

multi-stage Dockerfileを再現して切り分ける

multi-stage Dockerfileの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: builder stage
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: runtime stage
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: non-root
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: minimal context

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	builder stage	builder stage — 入力と前提を確認し、境界を曖昧にしない。
2	runtime stage	runtime stage — 値がどの順で変換されるかを追跡する。
3	non-root	non-root — 失敗を再現し、型またはログから原因を特定する。
4	minimal context	minimal context — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 runtime imageへbinary・assets・必要libraryだけを非rootで配置します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 02 ・ PRACTICE

手を動かす演習

docker historyとcontainer内のuser、files、healthを確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 02 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
docker build -t rustnote:local .
docker run --rm --entrypoint id rustnote:local
docker run --rm --entrypoint ls rustnote:local -la /app
docker run --rm -p 8080:8080 -e HOST=0.0.0.0 -e PORT=8080 rustnote:local
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ runtime imageへbinary・assets・必要libraryだけを非rootで配置します。
- ✓ multi-stage Dockerfileとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 03

Topcoat asset release bundle

UNIT 03 · CONCEPT

Topcoat asset release bundle

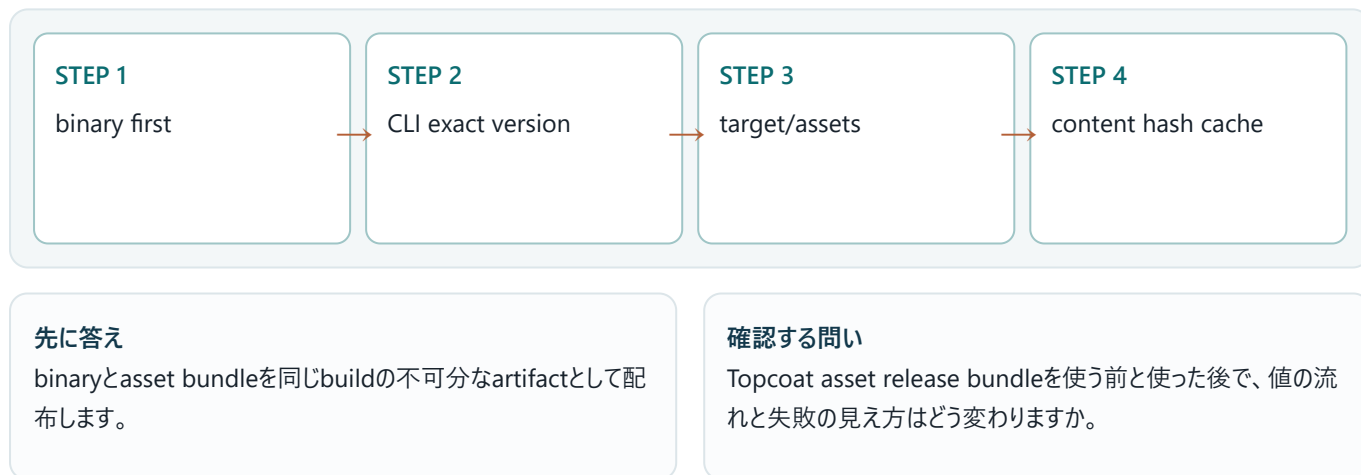
Topcoat 0.5.0はcompile済みbinary内のasset!参照をCLIが走査します。したがってrelease binaryを先にbuildし、その直後に同じsourceでasset bundleを作ります。

この単元の到達点

- binary first
- CLI exact version
- target/assets
- content hash cache

なぜバックエンドで必要か

cargo runだけでAssetBundle::loadを使うとbundleがなく起動失敗する場合があります。開発はtopcoat dev、本番はbuild→bundleです。



UNIT 03 · MENTAL MODEL

Topcoat asset release bundleを図でつかむ



Topcoat 0.5.0はcompile済みbinary内のasset!参照をCLIが走査します。したがってrelease binaryを先にbuildし、その直後に同じsourceでasset bundleを作ります。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認しま

す。

順序	観察対象	初心者が確認すること
1	binary first	binary first — 入力と前提を確認し、境界を曖昧にしない。
2	CLI exact version	CLI exact version — 値がどの順で変換されるかを追跡する。
3	target/assets	target/assets — 失敗を再現し、型またはログから原因を特定する。
4	content hash cache	content hash cache — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

Topcoat asset release bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 03 · MINIMUM CODE

まず動く最小コード

Topcoat asset release bundleだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
cargo build --locked --release
topcoat asset bundle --release
Get-ChildItem target\assets
# Docker COPY /app/target/assets /app/assets
```

● ● ● 実行コマンド

```
cargo build --locked --release
topcoat asset bundle --release
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 03 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>cargo build --locked --release</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>topcoat asset bundle --release</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code>Get-ChildItem target¥assets</code>	この行が受け取る値、作る値、副作用の有無を確認します。
<code># Docker COPY /app/target/assets /app/assets</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力値がbinary firstの条件を満たすか確認する。
- ✓ CLI exact versionを実行し、途中の型と状態を追う。
- ✓ target/assetsで失敗を成功経路から分離する。
- ✓ content hash cacheにより、出力と副作用を検証する。

型を声に出す Topcoat asset release bundleに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つを変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 03 · CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
manifest.toml
app-<hash>.css
Topcoat runtime assets...
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力値が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 03 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
# before/after each change
cargo build --locked --release
topcoat asset bundle --release
# manifest and generated filenames differ only for changed content
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

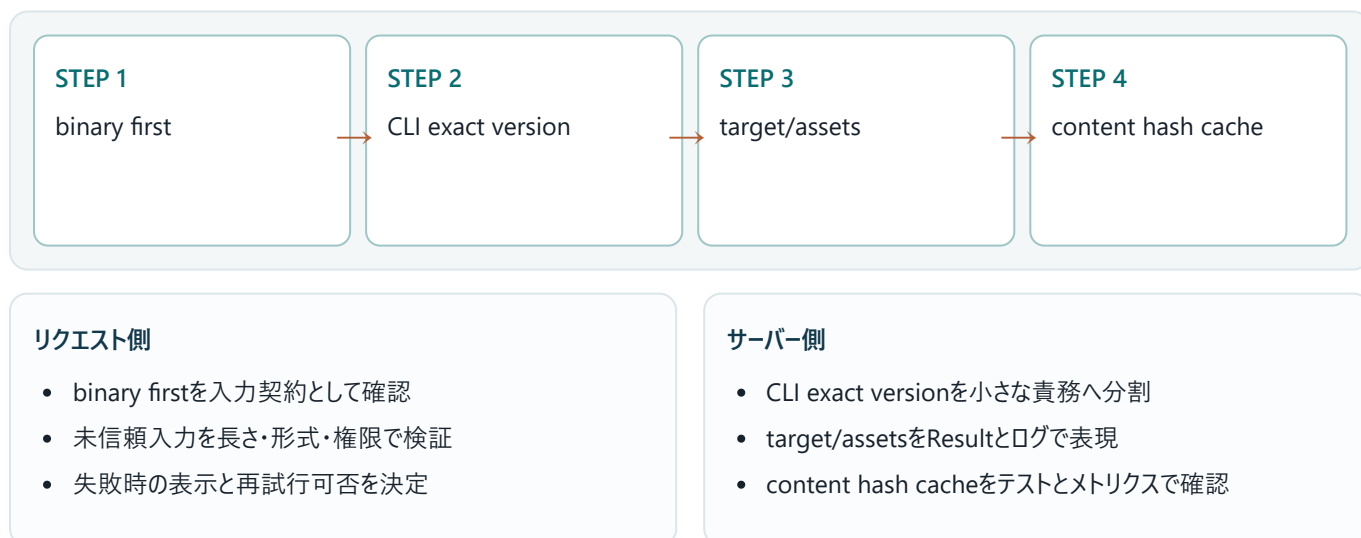
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 03 · BACKEND APPLICATION

バックエンドのどこで使うか

cargo runだけでAssetBundle::loadを使うとbundleがなく起動失敗する場合があります。開発はtopcoat dev、本番はbuild→bundleです。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 03・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	binary firstに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	CLI exact versionの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	target/assetsまたはcontent hash cacheの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 03・DEBUGGING

Topcoat asset release bundleを再現して切り分ける

Topcoat asset release bundleの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: binary first
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: CLI exact version
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: target/assets
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: content hash cache

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	binary first	binary first — 入力と前提を確認し、境界を曖昧にしない。
2	CLI exact version	CLI exact version — 値がどの順で変換されるかを追跡する。
3	target/assets	target/assets — 失敗を再現し、型またはログから原因を特定する。
4	content hash cache	content hash cache — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 binaryとasset bundleを同じbuildの不可分なartifactとして配布します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 03 ・ PRACTICE

手を動かす演習

CSS変更前後でhash filenameが変わり、HTML参照も新hashになることを確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 03 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
# before/after each change
cargo build --locked --release
topcoat asset bundle --release
# manifest and generated filenames differ only for changed content
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ binaryとasset bundleを同じbuildの不可分なartifactとして配布します。
- ✓ Topcoat asset release bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 04

fly launch・fly.toml・port

UNIT 04・CONCEPT

fly launch・fly.toml・port

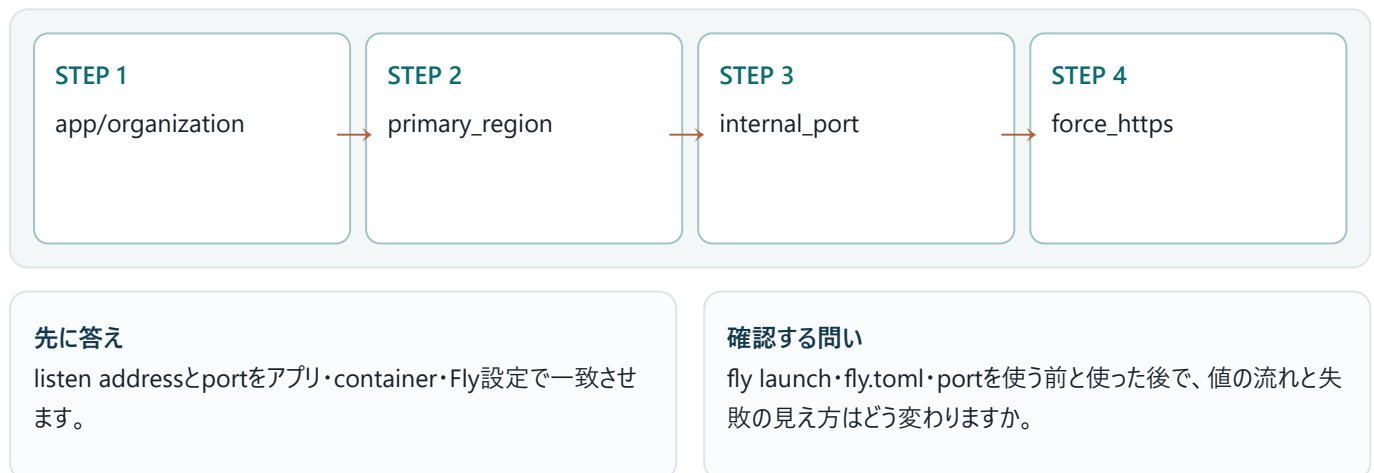
fly launch --no-deployで設定を生成し、アプリがHOST=0.0.0.0、PORT=8080で待ち受け、fly.toml internal_port=8080と一致させます。primary_regionをTursoや利用者に近づけます。

この単元の到達点

- app/organization
- primary_region
- internal_port
- force_https

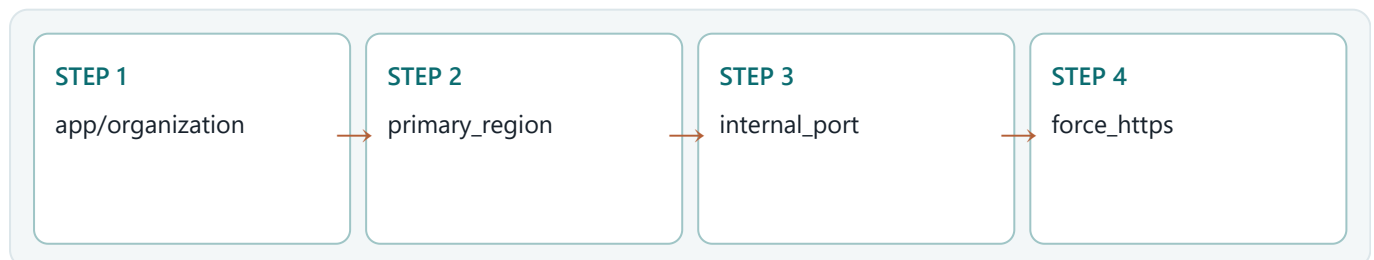
なぜバックエンドで必要か

127.0.0.1 bindやport不一致はdeploy成功でも到達不能になります。Docker local smokeで同じenvを先に試します。



UNIT 04・MENTAL MODEL

fly launch・fly.toml・portを図でつかむ



fly launch --no-deployで設定を生成し、アプリがHOST=0.0.0.0、PORT=8080で待ち受け、fly.toml internal_port=8080と一致させます。primary_regionをTursoや利用者に近づけます。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	app/organization	app/organization — 入力と前提を確認し、境界を曖昧にしない。
2	primary_region	primary_region — 値がどの順で変換されるかを追跡する。
3	internal_port	internal_port — 失敗を再現し、型またはログから原因を特定する。
4	force_https	force_https — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

fly launch・fly.toml・portとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 04 · MINIMUM CODE まず動く最小コード

fly launch・fly.toml・portだけに注目した最小例です。まず変更せず実行し、出力が一致した後に一行ずつ意味を確認します。

```
app="rustnote-example"
primary_region="nrt"
[env]
  HOST="0.0.0.0"
  PORT="8080"
[http_service]
  internal_port=8080
  force_https=true
  auto_stop_machines="stop"
  auto_start_machines=true
```

● ● ● 実行コマンド

```
fly launch --no-deploy
fly config validate
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 04 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
app="rustnote-example"	この行が受け取る値、作る値、副作用の有無を確認します。
primary_region="nrt"	この行が受け取る値、作る値、副作用の有無を確認します。
[env]	この行が受け取る値、作る値、副作用の有無を確認します。
HOST="0.0.0.0"	この行が受け取る値、作る値、副作用の有無を確認します。
PORT="8080"	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力app/organizationの条件を満たすか確認する。
- ✓ primary_regionを実行し、途中の型と状態を追う。
- ✓ internal_portで失敗を成功経路から分離する。
- ✓ force_httpsにより、出力と副作用を検証する。

型を声に出す fly launch・fly.toml・portに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 04・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

```
fly config validate → valid
local container listens 0.0.0.0:8080
```

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 04 · VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
Topcoat: HOST 0.0.0.0 PORT 8080
Docker: EXPOSE 8080 optional documentation
Fly: internal_port 8080
Health: GET http://machine:8080/health
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

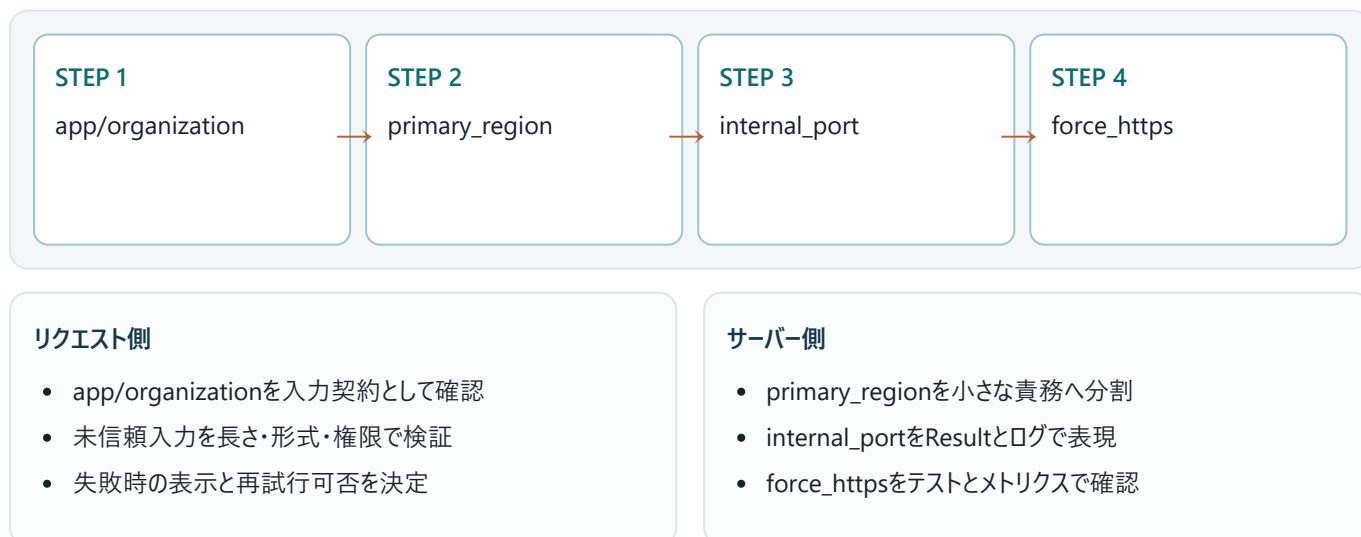
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 04 · BACKEND APPLICATION

バックエンドのどこで使うか

127.0.0.1 bindやport不一致はdeploy成功でも到達不能になります。Docker local smokeで同じenvを先に試します。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 04 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	app/organizationに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	primary_regionの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	internal_portまたはforce_httpsの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 04・DEBUGGING

fly launch・fly.toml・portを再現して切り分ける

fly launch・fly.toml・portの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: app/organization
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: primary_region
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: internal_port
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: force_https

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	app/organization	app/organization — 入力と前提を確認し、境界を曖昧にしない。
2	primary_region	primary_region — 値がどの順で変換されるかを追跡する。
3	internal_port	internal_port — 失敗を再現し、型またはログから原因を特定する。
4	force_https	force_https — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 listen addressとportをアプリ・container・Fly設定で一致させます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 04 · PRACTICE

手を動かす演習

fly.toml、Docker EXPOSE、Topcoat HOST/PORT、health URLの四か所を表にして一致確認してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 04 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
Topcoat: HOST 0.0.0.0 PORT 8080
Docker: EXPOSE 8080 optional documentation
Fly: internal_port 8080
Health: GET http://machine:8080/health
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ listen addressとportをアプリ・container・Fly設定で一致させます。
- ✓ fly launch・fly.toml・portとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 05

secrets・config・rotation

UNIT 05 ・ CONCEPT

secrets・config・rotation

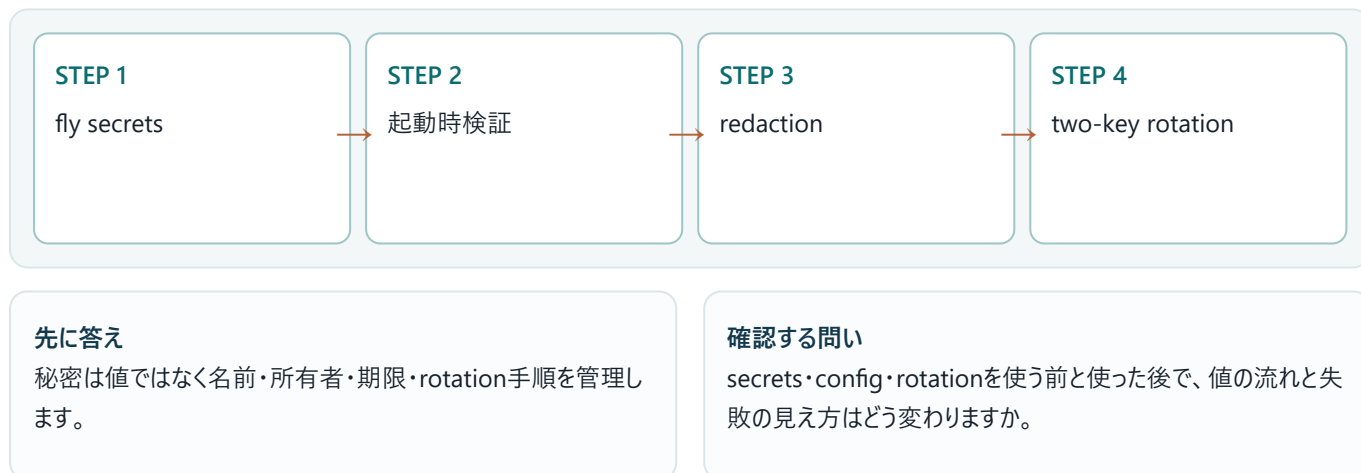
Turso token、session secret、外部API keyはimageやfly.tomlへ書かずfly secretsへ保存します。名前だけを文書化し、値をlogやshell historyへ残さず、rotationを二段階で行います。

この単元の到達点

- fly secrets
- 起動時検証
- redaction
- two-key rotation

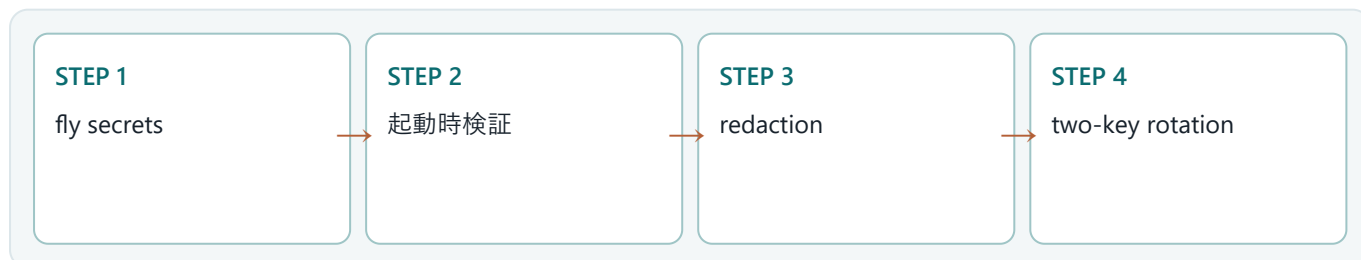
なぜバックエンドで必要か

secret更新はMachine再起動を伴います。新旧token共存→新token deploy・疎通→旧token revokeで切断時間を避けます。



UNIT 05 ・ MENTAL MODEL

secrets・config・rotationを図でつかむ



Turso token、session secret、外部API keyはimageやfly.tomlへ書かずfly secretsへ保存します。名前だけを文書化し、値をlogやshell historyへ残さず、rotationを二段階で行います。図では左から右へ処理を追います。各箱の入口では入力 の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	fly secrets	fly secrets — 入力と前提を確認し、境界を曖昧にしない。
2	起動時検証	起動時検証 — 値がどの順で変換されるかを追跡する。
3	redaction	redaction — 失敗を再現し、型またはログから原因を特定する。
4	two-key rotation	two-key rotation — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出来されるか」を自分の言葉で説明します。

30秒説明

secrets・config・rotationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 05・MINIMUM CODE まず動く最小コード

secrets・config・rotationだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fly secrets set TURSO_DATABASE_URL=libsql://...
fly secrets set TURSO_AUTH_TOKEN=<value>
fly secrets list
# 値はlistに表示されない
```

●●● 実行コマンド

```
fly secrets list
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 05・LINE BY LINE コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
fly secrets set TURSO_DATABASE_URL=libsql://...	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly secrets set TURSO_AUTH_TOKEN=<value>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly secrets list	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
# 値はlistに表示されない	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力がfly secretsの条件を満たすか確認する。
- ✓ 起動時検証を実行し、途中の型と状態を追う。
- ✓ redactionで失敗を成功経路から分離する。
- ✓ two-key rotationにより、出力と副作用を検証する。

型を声に出す secrets・config・rotationに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 05 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● 実行結果

```
NAME TURSO_DATABASE_URL Digest ...
NAME TURSO_AUTH_TOKEN Digest ...
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 05 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
phase1 app verifies old+new, issues new only
phase2 deploy new secret set and monitor
phase3 wait max cookie lifetime or force reauth
phase4 remove old, test revoked sessions
never log values
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

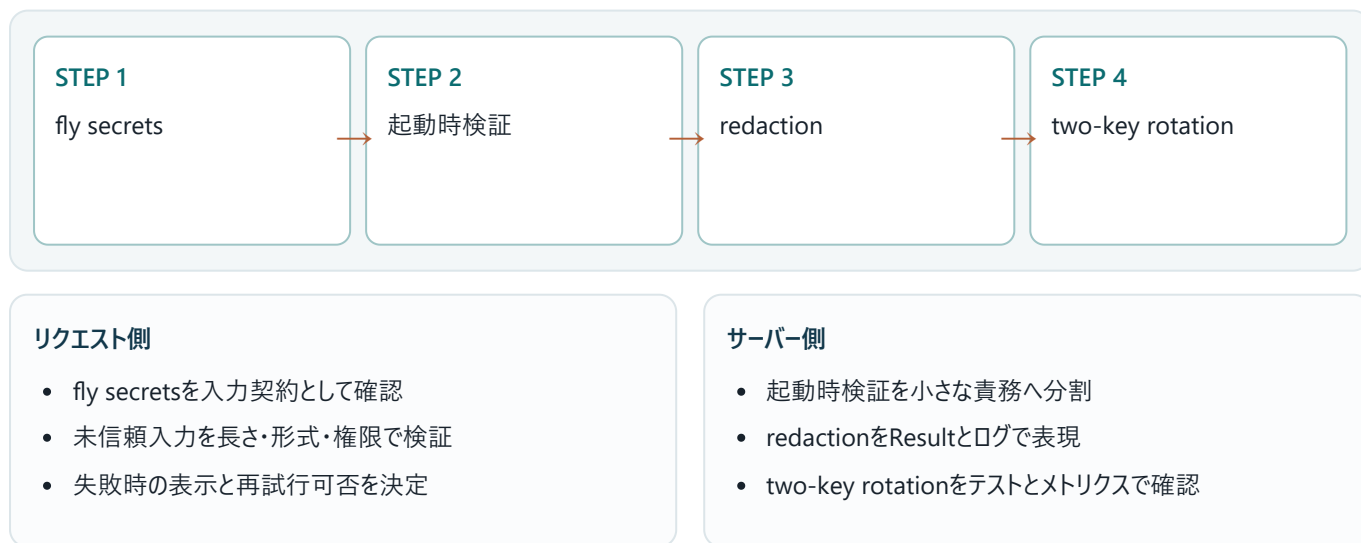
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 05 · BACKEND APPLICATION

バックエンドのどこで使うか

secret更新はMachine再起動を伴います。新旧token共存→新token deploy・疎通→旧token revokeで切断時間を避けます。



境界で守る不変条件

- ✓ 不正な入力副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 05 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	fly secretsに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	起動時検証の入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	redactionまたはtwo-key rotationの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 05・DEBUGGING

secrets・config・rotationを再現して切り分ける

secrets・config・rotationの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: fly secrets
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: 起動時検証
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: redaction
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: two-key rotation

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	fly secrets	fly secrets — 入力と前提を確認し、境界を曖昧にしない。
2	起動時検証	起動時検証 — 値がどの順で変換されるかを追跡する。
3	redaction	redaction — 失敗を再現し、型またはログから原因を特定する。
4	two-key rotation	two-key rotation — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 秘密は値ではなく名前・所有者・期限・rotation手順を管理します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 05 ・ PRACTICE

手を動かす演習

session signing secretを新旧二つ読める移行期間でrotateする計画を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 05 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
phase1 app verifies old+new, issues new only
phase2 deploy new secret set and monitor
phase3 wait max cookie lifetime or force reauth
phase4 remove old, test revoked sessions
never log values
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ 秘密は値ではなく名前・所有者・期限・rotation手順を管理します。
- ✓ secrets・config・rotationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 06

health check・startup・readiness

UNIT 06・CONCEPT

health check・startup・readiness

Flyのservice checkはtrafficを送れるか判断します。process生存のhealthとDB依存のreadyを分け、startupにmigrationが必要ならgrace periodとtimeoutを現実的に設定します。

この単元の到達点

- service check
- grace period
- interval/timeout
- liveness/readiness

なぜバックエンドで必要か

DB一時障害でMachine再起動loopにしないよう、何をcheckへ含めるかを決めます。ready失敗はtraffic除外、health失敗はprocess問題に寄せます。



UNIT 06・MENTAL MODEL

health check・startup・readinessを図でつかむ



Flyのservice checkはtrafficを送れるか判断します。process生存のhealthとDB依存のreadyを分け、startupにmigrationが必要ならgrace periodとtimeoutを現実的に設定します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の

型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	service check	service check — 入力と前提を確認し、境界を曖昧にしない。
2	grace period	grace period — 値がどの順で変換されるかを追跡する。
3	interval/timeout	interval/timeout — 失敗を再現し、型またはログから原因を特定する。
4	liveness/readiness	liveness/readiness — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

health check・startup・readinessとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 06・MINIMUM CODE

まず動く最小コード

health check・startup・readinessだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
[[http_service.checks]]
  grace_period="10s"
  interval="15s"
  timeout="2s"
  method="GET"
  path="/health"
```

●●● 実行コマンド

```
fly checks list
fly status
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 06・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
[[http_service.checks]]	この行が受け取る値、作る値、副作用の有無を確認します。
grace_period="10s"	この行が受け取る値、作る値、副作用の有無を確認します。
interval="15s"	この行が受け取る値、作る値、副作用の有無を確認します。
timeout="2s"	この行が受け取る値、作る値、副作用の有無を確認します。
method="GET"	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 service check の条件を満たすか確認する。
- ✓ grace period を実行し、途中の型と状態を追う。
- ✓ interval/timeout で失敗を成功経路から分離する。
- ✓ liveness/readiness により、出力と副作用を検証する。

型を声に出す health check・startup・readiness に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 06・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

fly checks list → passing
GET /health → 200 ok

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTP ステータス、標準エラー、ログ、DB の行数など、画面に現れない観測点も合わせて確認します。

UNIT 06・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
assert_eq!(health().await.status(),200);fake_db.fail_with_timeout();assert_eq!(
ready().await.status(),503);assert_eq!(health().await.status(),200);
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

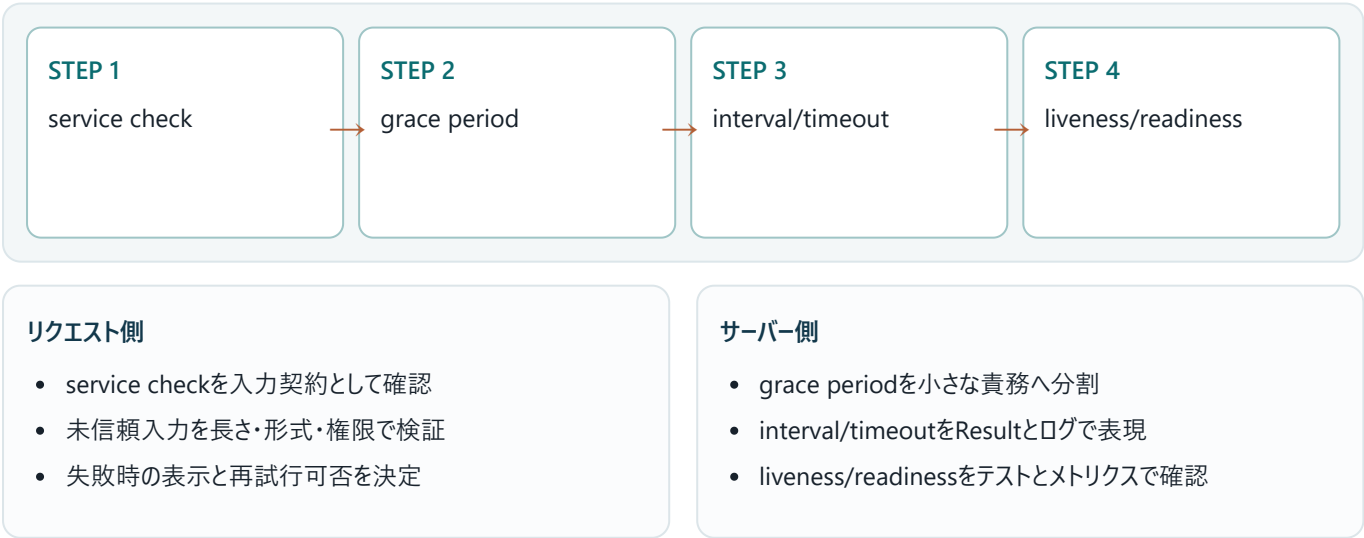
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 06・BACKEND APPLICATION

バックエンドのどこで使うか

DB一時障害でMachine再起動loopにしないよう、何をcheckへ含めるかを決めます。ready失敗はtraffic除外、health失敗はprocess問題に寄せます。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。

- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 06・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	service checkに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	grace periodの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	interval/timeoutまたはliveness/readinessの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 06・DEBUGGING

health check・startup・readinessを再現して切り分ける

health check・startup・readinessの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: service check
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: grace period
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: interval/timeout
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: liveness/readiness

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	service check	service check — 入力と前提を確認し、境界を曖昧にしない。
2	grace period	grace period — 値がどの順で変換されるかを追跡する。
3	interval/timeout	interval/timeout — 失敗を再現し、型またはログから原因を特定する。
4	liveness/readiness	liveness/readiness — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 checkの目的をtraffic制御とprocess生存で分け、依存障害時の再起動loopを防ぎます。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 06 ・ PRACTICE

手を動かす演習

healthが200、readyがDB timeoutで503になるintegration testを書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 06 ・ SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
assert_eq!(health().await.status(), 200); fake_db.fail_with_timeout(); assert_eq!(ready().await.status(), 503); assert_eq!(health().await.status(), 200);
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ checkの目的をtraffic制御とprocess生存で分け、依存障害時の再起動loopを防ぎます。
- ✓ health check・startup・readinessとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 07

migration deploy順・release command

UNIT 07 · CONCEPT

migration deploy順・release command

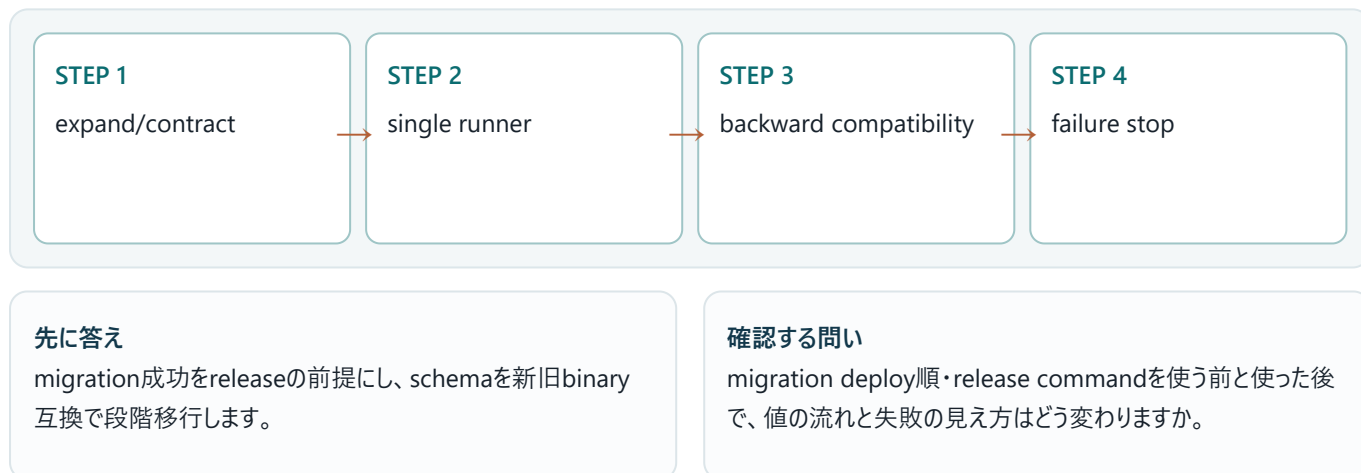
schema変更は新旧binary互換を保つexpand/contractで行います。migrationを複数Machineが同時実行しないよう、release commandまたは排他lockを使い、失敗時は新releaseをtrafficへ出しません。

この単元の到達点

- expand/contract
- single runner
- backward compatibility
- failure stop

なぜバックエンドで必要か

SQLite/libSQLのALTER制約を確認し、大表のindex作成やbackfillを一度に行わず負荷とlockを測ります。



UNIT 07 · MENTAL MODEL

migration deploy順・release commandを図でつかむ



schema変更は新旧binary互換を保つexpand/contractで行います。migrationを複数Machineが同時実行しないよう、release commandまたは排他lockを使い、失敗時は新releaseをtrafficへ出しません。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	expand/contract	expand/contract — 入力と前提を確認し、境界を曖昧にしない。
2	single runner	single runner — 値がどの順で変換されるかを追跡する。
3	backward compatibility	backward compatibility — 失敗を再現し、型またはログから原因を特定する。
4	failure stop	failure stop — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

migration deploy順・release commandとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 07 · MINIMUM CODE まず動く最小コード

migration deploy順・release commandだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
[deploy]
  release_command="/app/rustnote migrate"

# deploy 1: add nullable column/index
# deploy 2: app dual-read/write
# deploy 3: backfill/verify
# deploy 4: drop old in later release
```

● ● ● 実行コマンド

cargo run

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 07 · LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
[deploy]	この行が受け取る値、作る値、副作用の有無を確認します。
release_command="/app/rustnote migrate"	この行が受け取る値、作る値、副作用の有無を確認します。
# deploy 1: add nullable column/index	この行が受け取る値、作る値、副作用の有無を確認します。
# deploy 2: app dual-read/write	この行が受け取る値、作る値、副作用の有無を確認します。
# deploy 3: backfill/verify	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力 expand/contract の条件を満たすか確認する。
- ✓ single runner を実行し、途中の型と状態を追う。
- ✓ backward compatibility で失敗を成功経路から分離する。
- ✓ failure stop により、出力と副作用を検証する。

型を声に出す migration deploy 順・release command に関する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 07・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

```
release_command success → Machines roll
failure → deploy stops before traffic
```

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 07・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
D1 add new_name nullable
D2 write old+new/read fallback
Backfill in batches with metrics
D3 read new only after coverage 100%
D4 stop old write
D5 later remove old after rollback window
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

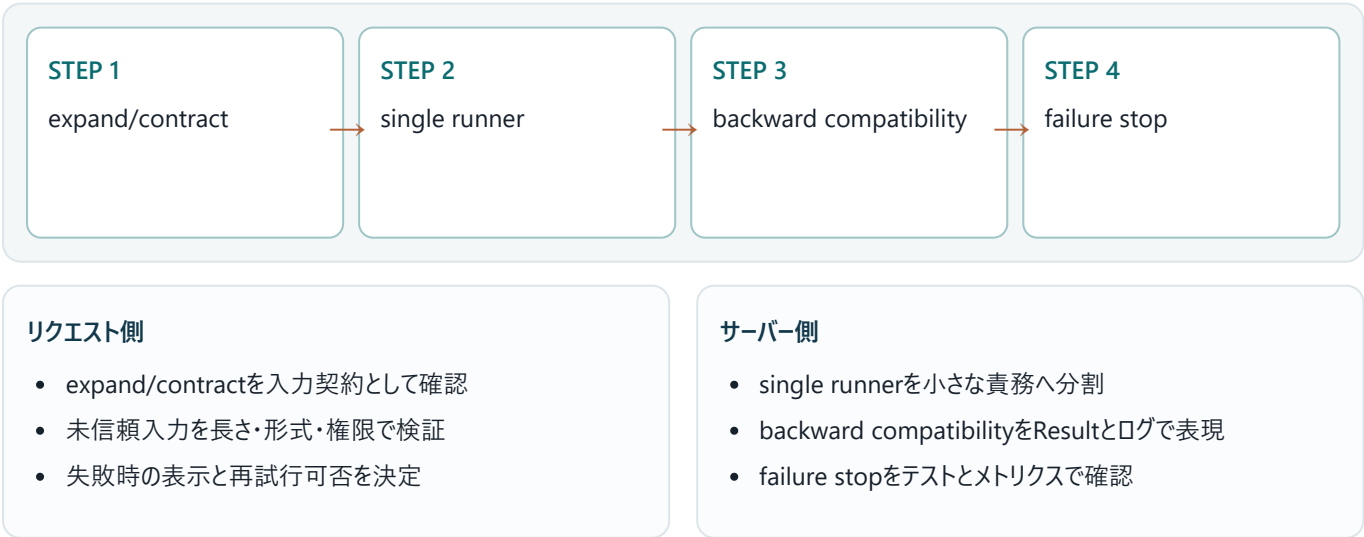
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 07・BACKEND APPLICATION

バックエンドのどこで使うか

SQLite/libSQLのALTER制約を確認し、大表のindex作成やbackfillを一度に行わず負荷とlockを測ります。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 07 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	expand/contractに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	single runnerの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	backward compatibilityまたはfailure stopの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

```
error: example failed
help: 最初の原因行と型の差を確認してください
```

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 07・DEBUGGING

migration deploy順・release commandを再現して切り分ける

migration deploy順・release commandの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: expand/contract
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: single runner
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: backward compatibility
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: failure stop

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	expand/contract	expand/contract — 入力と前提を確認し、境界を曖昧にしない。
2	single runner	single runner — 値がどの順で変換されるかを追跡する。
3	backward compatibility	backward compatibility — 失敗を再現し、型またはログから原因を特定する。
4	failure stop	failure stop — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 migration成功をreleaseの前提にし、schemaを新旧binary互換で段階移行します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 07 · PRACTICE

手を動かす演習

column renameを直接行わない四deployのexpand/contract計画を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 07 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
D1 add new_name nullable
D2 write old+new/read fallback
Backfill in batches with metrics
D3 read new only after coverage 100%
D4 stop old write
D5 later remove old after rollback window
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ migration成功をreleaseの前提にし、schemaを新旧binary互換で段階移行します。
- ✓ migration deploy順・release commandとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 08

deploy・logs・smoke・custom domain

UNIT 08 ・ CONCEPT

deploy・logs・smoke・custom domain

fly deploy後にstatusだけで成功とせず、checks、logs、version、health、auth、CRUD、assetをsmokeします。custom domainではDNSとcertificate状態を確認します。

この単元の到達点

- fly deploy
- checks/logs
- post-deploy smoke
- DNS/TLS

なぜバックエンドで必要か

smokeは専用test account/dataを使い、本番利用者データを変更しません。Set-Cookie、HTTPS redirect、security headersも確認します。



先に答え

deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。

確認する問い

deploy・logs・smoke・custom domainを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

UNIT 08 ・ MENTAL MODEL

deploy・logs・smoke・custom domainを図でつかむ



fly deploy後にstatusだけで成功とせず、checks、logs、version、health、auth、CRUD、assetをsmokeします。custom domainではDNSとcertificate状態を確認します。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認

認めます。

順序	観察対象	初心者が確認すること
1	fly deploy	fly deploy — 入力と前提を確認し、境界を曖昧にしない。
2	checks/logs	checks/logs — 値がどの順で変換されるかを追跡する。
3	post-deploy smoke	post-deploy smoke — 失敗を再現し、型またはログから原因を特定する。
4	DNS/TLS	DNS/TLS — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

deploy・logs・smoke・custom domainとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 08・MINIMUM CODE

まず動く最小コード

deploy・logs・smoke・custom domainだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fly deploy
fly status
fly checks list
fly logs
curl -i https://<app>.fly.dev/health
curl -i https://<app>.fly.dev/assets/<hash>.css
```

●●● 実行コマンド

```
fly deploy --strategy rolling
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力

ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力

標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 08・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
<code>fly deploy</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>fly status</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>fly checks list</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>fly logs</code>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
<code>curl -i https://<app>.fly.dev/health</code>	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力が入力型 fly deploy の条件を満たすか確認する。
- ✓ checks/logs を実行し、途中の型と状態を追う。
- ✓ post-deploy smoke で失敗を成功経路から分離する。
- ✓ DNS/TLS により、出力と副作用を検証する。

型を声に出す deploy・logs・smoke・custom domain に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 08 ・ CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

● ● ● http://localhost:3000/learn

Rust Backend Lab

実行する

deploy・logs・smoke・custom domain

公開HTTPS URLでCSS付きRustNoteが開き、registerからlogoutまでsmoke accountで確認できます。

観察ポイント

- fly deploy
- checks/logs
- post-deploy smoke

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 08 ・ VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
0-1m checks/version/health
1-3m CSS/register/login/create/delete test note
3-5m logs/5xx/latency/Turso errors
Stop+rollback: check fail, auth fail, data corruption, elevated 5xx
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 08 · BACKEND APPLICATION

バックエンドのどこで使うか

smokeは専用test account/dataを使い、本番利用者データを変更しません。Set-Cookie、HTTPS redirect、security headersも確認します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 08 · FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	fly deployに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	checks/logsの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	post-deploy smokeまたはDNS/TLSの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

● ● ● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 08・DEBUGGING

deploy・logs・smoke・custom domainを再現して切り分ける

deploy・logs・smoke・custom domainの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: fly deploy
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: checks/logs
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: post-deploy smoke
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: DNS/TLS

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	fly deploy	fly deploy — 入力と前提を確認し、境界を曖昧にしない。
2	checks/logs	checks/logs — 値がどの順で変換されるかを追跡する。
3	post-deploy smoke	post-deploy smoke — 失敗を再現し、型またはログから原因を特定する。
4	DNS/TLS	DNS/TLS — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 08 ・ PRACTICE

手を動かす演習

deploy後5分のsmoke checklistと失敗時の停止条件を書いてください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 08 · SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
0-1m checks/version/health
1-3m CSS/register/login/create/delete test note
3-5m logs/5xx/latency/Turso errors
Stop+rollback: check fail, auth fail, data corruption, elevated 5xx
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。
- ✓ deploy・logs・smoke・custom domainとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 09

scale・region・cost・Turso latency

UNIT 09・CONCEPT

scale・region・cost・Turso latency

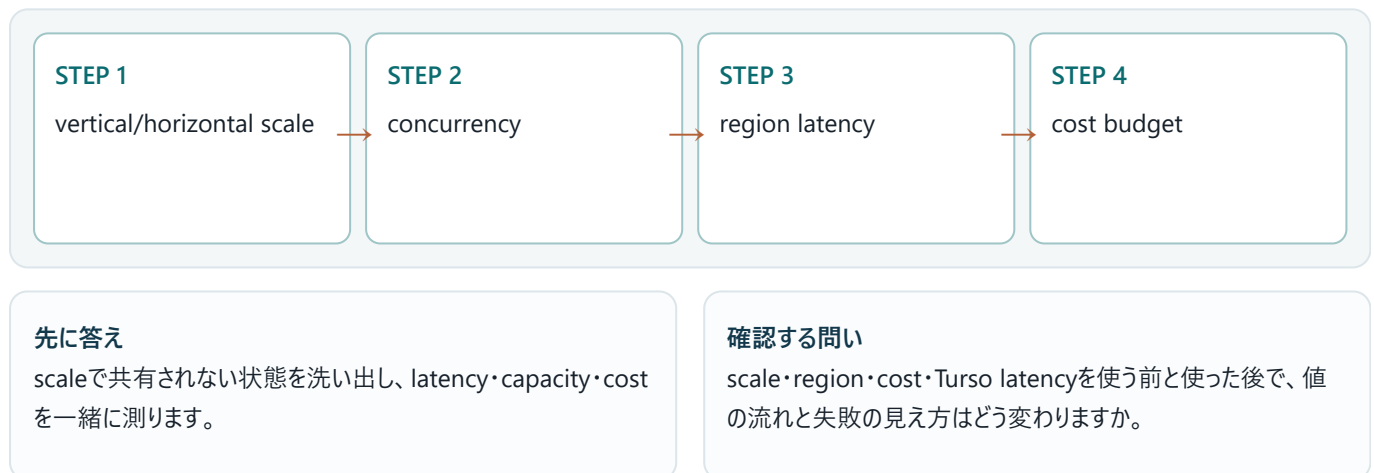
Machine数、size、auto stop、concurrency、regionはlatencyとcostを変えます。Rustの低memoryだけを前提にせず、Argon2 CPU、DB接続、idle起動時間を測ります。

この単元の到達点

- vertical/horizontal scale
- concurrency
- region latency
- cost budget

なぜバックエンドで必要か

複数MachineでもsessionをDBへ置くためsticky session不要です。in-memory rate limitやjob lockは分散状態として再設計します。



UNIT 09・MENTAL MODEL

scale・region・cost・Turso latencyを図でつかむ



Machine数、size、auto stop、concurrency、regionはlatencyとcostを変えます。Rustの低memoryだけを前提にせず、Argon2 CPU、DB接続、idle起動時間を測ります。図では左から右へ処理を追います。各箱の入口では入力型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	vertical/horizontal scale	vertical/horizontal scale — 入力と前提を確認し、境界を曖昧にしない。
2	concurrency	concurrency — 値がどの順で変換されるかを追跡する。
3	region latency	region latency — 失敗を再現し、型またはログから原因を特定する。
4	cost budget	cost budget — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

scale・region・cost・Turso latencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 09・MINIMUM CODE

まず動く最小コード

scale・region・cost・Turso latencyだけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fly scale show
fly machine list
fly scale count 2
fly scale vm shared-cpu-1x --memory 512
```

● ● ● 実行コマンド

```
fly scale show
fly dashboard
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順に進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 09・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
fly scale show	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly machine list	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly scale count 2	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly scale vm shared-cpu-1x --memory 512	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。

実行時に起きる順序

- ✓ 入力vertical/horizontal scaleの条件を満たすか確認する。
- ✓ concurrencyを実行し、途中の型と状態を追う。
- ✓ region latencyで失敗を成功経路から分離する。
- ✓ cost budgetにより、出力と副作用を検証する。

型を声に出す scale・region・cost・Turso latencyに関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値の一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 09・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

2 Machines in nrt
health passing
monthly estimate updated

見える結果
端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと
型検査を通過した後、入力が処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 09・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
local: memory cache/semaphore/temp/metrics buffer
Turso: users/notes/sessions/jobs/idempotency
external: DNS/TLS/email
2台時にlocal counterをglobal認可へ使わない
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

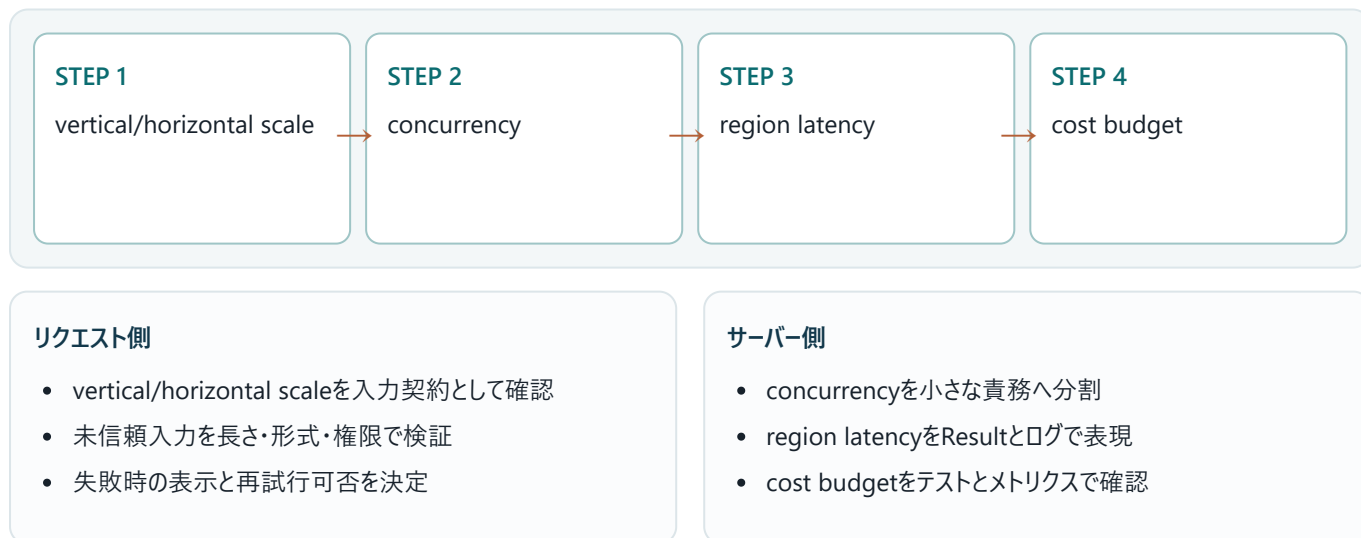
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 09 · BACKEND APPLICATION

バックエンドのどこで使うか

複数MachineでもsessionをDBへ置くためsticky session不要です。in-memory rate limitやjob lockは分散状態として再設計します。



境界で守る不変条件

- ✓ 不正な入力は副作用より前に拒否する。
- ✓ 他人のデータへアクセスさせない。
- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 09・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	vertical/horizontal scaleに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	concurrencyの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	region latencyまたはcost budgetの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 09・DEBUGGING

scale・region・cost・Turso latencyを再現して切り分ける

scale・region・cost・Turso latencyの問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: vertical/horizontal scale
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: concurrency
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: region latency
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: cost budget

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	vertical/horizontal scale	vertical/horizontal scale — 入力と前提を確認し、境界を曖昧にしない。
2	concurrency	concurrency — 値がどの順で変換されるかを追跡する。
3	region latency	region latency — 失敗を再現し、型またはログから原因を特定する。
4	cost budget	cost budget — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 scaleで共有されない状態を洗い出し、latency・capacity・costを一緒に測ります。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 09 ・ PRACTICE

手を動かす演習

1台→2台で変わる状態をprocess local、Turso shared、外部serviceに分類してください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 09・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

```
local: memory cache/semaphore/temp/metrics buffer
Turso: users/notes/sessions/jobs/idempotency
external: DNS/TLS/email
2台時にlocal counterをglobal認可へ使わない
```

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ scaleで共有されない状態を洗い出し、latency・capacity・costを一緒に測ります。
- ✓ scale・region・cost・Turso latencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

UNIT 10

rollback・backup・disaster recovery・更新

UNIT 10・CONCEPT

rollback・backup・disaster recovery・更新

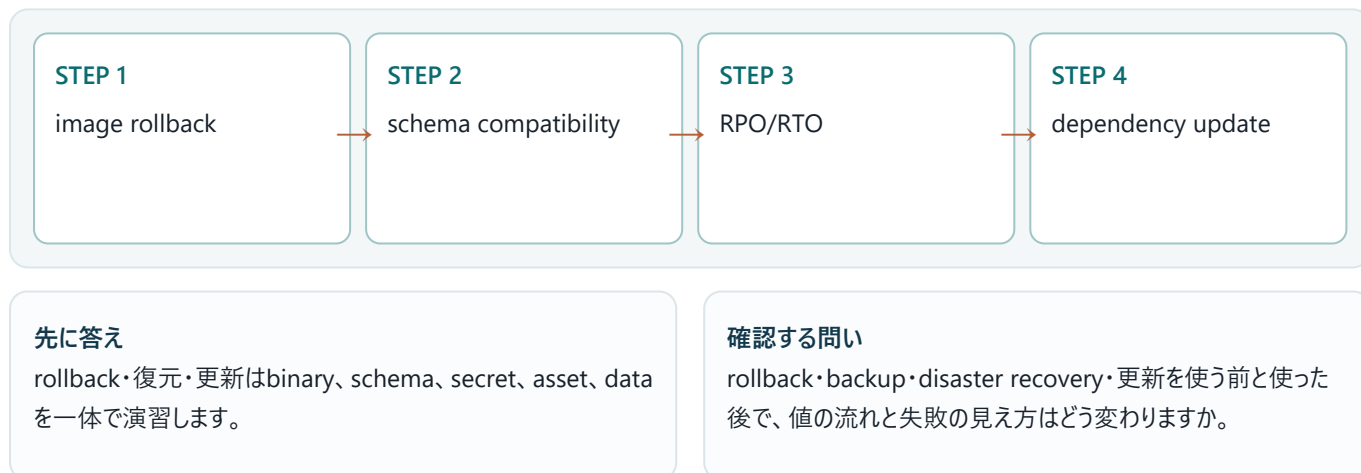
rollbackは前binaryへ戻すだけでなくschema互換、secret、job、assetを含みます。RPO/RTOを決め、Turso復元、Fly image rollback、token revokeを定期演習します。

この単元の到達点

- image rollback
- schema compatibility
- RPO/RTO
- dependency update

なぜバックエンドで必要か

Topcoatやlibsql更新は一度にまとめず、changelog、compile、test、bundle、migration、loadを確認し、rollback windowを残します。



UNIT 10・MENTAL MODEL

rollback・backup・disaster recovery・更新を図でつかむ



rollbackは前binaryへ戻すだけでなくschema互換、secret、job、assetを含みます。RPO/RTOを決め、Turso復元、Fly image rollback、token revokeを定期演習します。図では左から右へ処理を追います。各箱の入口では入力の型と信頼度、出口では結果の型と副作用を確認します。

順序	観察対象	初心者が確認すること
1	image rollback	image rollback — 入力と前提を確認し、境界を曖昧にしない。
2	schema compatibility	schema compatibility — 値がどの順で変換されるかを追跡する。
3	RPO/RTO	RPO/RTO — 失敗を再現し、型またはログから原因を特定する。
4	dependency update	dependency update — テストで期待結果を固定し、変更後も守る。

暗記しない 図の矢印を指で追い、「誰が値を持つか」「どこで失敗するか」「何が出力されるか」を自分の言葉で説明します。

30秒説明

rollback・backup・disaster recovery・更新とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

UNIT 10・MINIMUM CODE

まず動く最小コード

rollback・backup・disaster recovery・更新だけに注目した最小例です。まず変更せず実行し、出力が一致した後で一行ずつ意味を確認します。

```
fly releases
fly deploy --image <previous-image-digest>
fly checks list
# verify schema compatible and assets match binary
# restore drill uses isolated DB, never overwrites production
```

● ● ● 実行コマンド

```
fly releases
fly logs
```

写経の順序 ①そのまま入力 ②保存 ③実行 ④出力を比較 ⑤1か所だけ変更、の順で進めます。複数箇所を同時に変えないことが最短のデバッグ方法です。

入力
ソースコード、設定、利用者入力のうち、この例が受け取る値を確認します。

期待する出力
標準出力、HTTPレスポンス、DB更新のどれが成果かを区別します。

UNIT 10・LINE BY LINE

コードを行ごとに読む

構文の名前だけでなく、値の流れと責務の境界を読み取ります。次の表は、前ページのコードで特に重要な行を「何をするか」と「なぜ必要か」に分けたものです。

コード／場所	役割と理由
fly releases	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly deploy --image <previous-image-digest>	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
fly checks list	ツールの状態を変更・確認するコマンドです。実行ディレクトリと対象環境を先に確認します。
# verify schema compatible and assets match binary	この行が受け取る値、作る値、副作用の有無を確認します。
# restore drill uses isolated DB, never overwrites production	この行が受け取る値、作る値、副作用の有無を確認します。

実行時に起きる順序

- ✓ 入力image rollbackの条件を満たすか確認する。
- ✓ schema compatibilityを実行し、途中の型と状態を追う。
- ✓ RPO/RTOで失敗を成功経路から分離する。
- ✓ dependency updateにより、出力と副作用を検証する。

型を声に出す rollback・backup・disaster recovery・更新に関係する入力型、途中の型、最終的な出力型を順番に書き出します。

変更実験

入力値を一つ変更し、コンパイル結果または実行結果がなぜ変わるかを説明します。

UNIT 10・CODE TO OUTPUT

このコードは何を表示するか

コードと結果を必ず同じ見開きのつもりで比較します。結果が一致しない場合は、入力値、実行ディレクトリ、環境変数、バージョンの順に確認します。

●●● 実行結果

previous digest deployed
health/auth/CRUD smoke passing

見える結果

端末またはブラウザに、期待した値と状態が表示されます。

内部で起きたこと

型検査を通過した後、入力処理され、境界を越えて出力へ変換されます。

表示だけで合否を決めない HTTPステータス、標準エラー、ログ、DBの行数など、画面に現れない観測点も合わせて確認します。

UNIT 10・VARIATION

一段実務寄りに書き換える

最小例へ検証とエラー処理を加え、呼び出し側が成功と失敗を区別できる形にします。

```
DB: stop writes→preserve evidence→select restore point→restore isolated→validate→switch→reconcile→postmortem
Dependency: assess exploitability→pin/fix branch→test+SBOM→canary→monitor→rollout/rollback
```

変更点

- 固定値を引数または設定へ移す。
- 失敗を握り潰さずResultで返す。
- 観測可能なログまたはテストを加える。
- 境界に型と検証を置く。

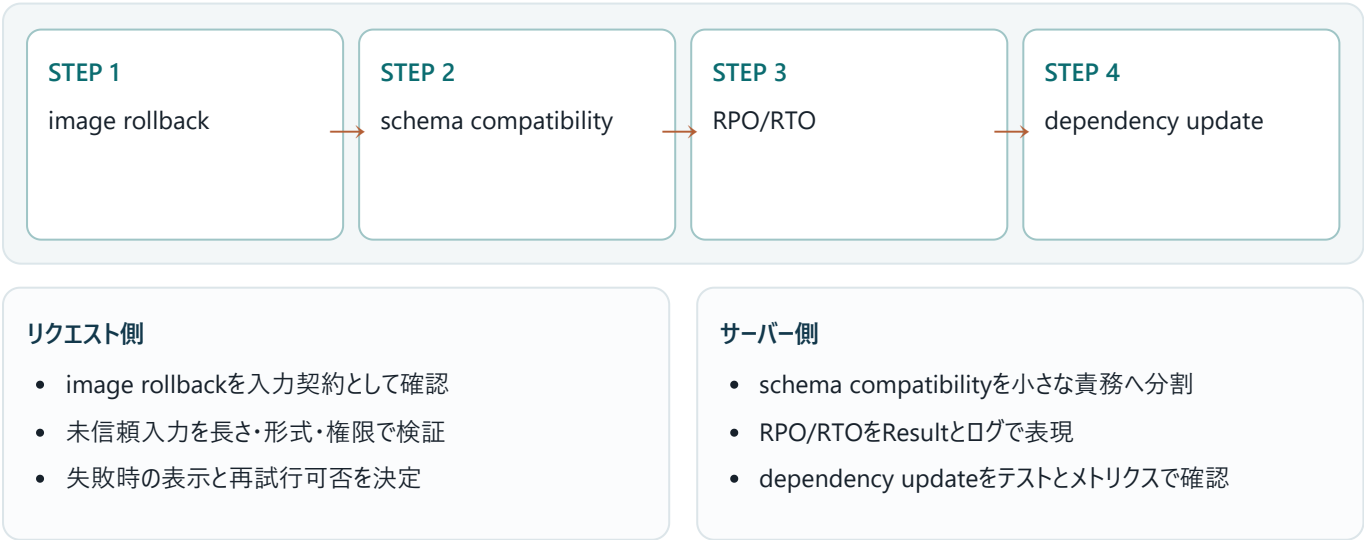
設計判断 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

観点	最小例	実務版
読みやすさ	処理を一か所に集める	責務ごとに関数・型へ分ける
失敗	固定値または即時終了	Result、ログ、利用者向けメッセージを分離
検証	目視	テストと観測値で再現可能にする

UNIT 10 · BACKEND APPLICATION

バックエンドのどこで使うか

Topcoatやlibsql更新は一度にまとめず、changelog、compile、test、bundle、migration、loadを確認し、rollback windowを残します。



- ✓ 同じ操作の再実行で破損させない。
- ✓ 失敗しても秘密情報をレスポンスやログへ出さない。

信頼境界 ブラウザ、環境変数、DB、外部APIから来る値は、Rustの型に入っただけでは安全になりません。形式、長さ、権限、期限を境界で検証します。

ログに残すもの

秘密値そのものは残さず、request_id、処理名、結果、件数、所要時間、エラー分類を構造化して残します。

UNIT 10・FAILURE

よくある失敗を先に見る

症状	原因	直し方
コンパイルできない	image rollbackに関する型・所有権・featureの不一致	最初のerrorとexpected/foundを読み、最小例へ戻す
結果が違う	schema compatibilityの入力または実行順序が想定と違う	入力、環境変数、現在ディレクトリを表示して比較する
本番だけ失敗	RPO/RTOまたはdependency updateの環境差	バージョン、秘密、ネットワーク、DB状態を順に照合する

●●● 失敗時の出力例

error: example failed
help: 最初の原因行と型の差を確認してください

エラーの読み方 最初の原因行 → エラーコード → expected / found → help の順に読みます。末尾の派生エラーから直し始めると遠回りになります。

切り分け

- ✓ エラー全文と再現コマンドを保存する。
- ✓ 最小入力で同じ症状が出るか確認する。
- ✓ 型、バージョン、環境変数、外部状態を一つずつ固定する。
- ✓ 原因を直したら失敗を再現するテストを追加する。

UNIT 10・DEBUGGING

rollback・backup・disaster recovery・更新を再現して切り分ける

rollback・backup・disaster recovery・更新の問題を、環境・入力・型・副作用の順に切り分けます。

- 1. STEP 1 — エラー全文と再現コマンドを保存する。対象: image rollback
- 2. STEP 2 — 最小入力で同じ症状が出るか確認する。対象: schema compatibility
- 3. STEP 3 — 型、バージョン、環境変数、外部状態を一つずつ固定する。対象: RPO/RTO
- 4. STEP 4 — 原因を直したら失敗を再現するテストを追加する。対象: dependency update

```
RUST_BACKTRACE=1 cargo run
```

この単元で観測する値

順	観察点	確認内容
1	image rollback	image rollback — 入力と前提を確認し、境界を曖昧にしない。
2	schema compatibility	schema compatibility — 値がどの順で変換されるかを追跡する。
3	RPO/RTO	RPO/RTO — 失敗を再現し、型またはログから原因を特定する。
4	dependency update	dependency update — テストで期待結果を固定し、変更後も守る。

この単元の終了条件 rollback・復元・更新はbinary、schema、secret、asset、dataを一体で演習します。原因を一文で説明し、同じ失敗を再現するテストを残せたら完了です。

UNIT 10 ・ PRACTICE

手を動かす演習

DB誤削除と脆弱依存更新の二つのrunbookを作ってください。

制約

- コンパイル警告を残さない。
- 正常系と失敗系を両方用意する。
- 秘密値や個人情報をログへ出さない。
- 外部入力を副作用の前に検証する。

進め方

1. 期待する入力と出力を紙に書く。
2. 前ページまでの最小コードを複製する。
3. 一度に一つだけ変更し、毎回実行する。
4. 正常系と失敗系を少なくとも一つずつ確認する。
5. 最後にフォーマット、Lint、テストを実行する。

● ● ● 確認コマンド

```
cargo fmt --check
cargo clippy -- -D warnings
cargo test
```

ヒント 最初から完成形を書かず、まず正常系を一つ通し、その後に失敗系を追加します。

採点基準

- ✓ コードが再現可能なコマンドで動く。
- ✓ 出力を自分の言葉で説明できる。
- ✓ 失敗時にpanicせず原因を返せる。
- ✓ 少なくとも一つの自動テストがある。

UNIT 10・SOLUTION

演習の解答例と考え方

解答の方針

入力を境界で検証し、処理を小さな関数へ分け、成功と失敗を型で返す方針にします。

DB: stop writes→preserve evidence→select restore point→restore isolated→validate→switch→reconcile→postmortem
Dependency: assess exploitability→pin/fix branch→test+SBOM→canary→monitor→rollout/rollback

解答例の読み方

- 入力検証を処理の冒頭へ置く。
- 成功値とエラーを呼び出し側が分岐できる型で返す。
- 副作用を小さな範囲へ閉じ込める。
- 変更後にfmt、clippy、testを通す。

別解を歓迎 出力と不変条件を満たし、失敗を説明できるなら別の実装も正解です。短さより、責務とエラーの場所が読み取れることを優先します。

単元チェック

- ✓ rollback・復元・更新はbinary、schema、secret、asset、dataを一体で演習します。
- ✓ rollback・backup・disaster recovery・更新とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。
- ✓ 短いコードより、入力・成功・失敗の型が呼び出し側から読める設計を優先します。

CAPSTONE

RustNoteをFly.ioへ公開し、rollback演習まで完了する

10単元を一つの成果物へ統合します。動く結果だけでなく、途中のエラー、設計判断、確認コマンドも提出物です。

順	使う単元	統合する判断
01	release build・reproducibility	source commit、lock、toolchain、artifact digestを一組で記録します。
02	multi-stage Dockerfile	runtime imageへbinary・assets・必要libraryだけを非rootで配置します。
03	Topcoat asset release bundle	binaryとasset bundleを同じbuildの不可分なartifactとして配布します。
04	fly launch・fly.toml・port	listen addressとportをアプリ・container・Fly設定で一致させます。
05	secrets・config・rotation	秘密は値ではなく名前・所有者・期限・rotation手順を管理します。
06	health check・startup・readiness	checkの目的をtraffic制御とprocess生存で分け、依存障害時の再起動loopを防ぎます。
07	migration deploy順・release command	migration成功をreleaseの前提にし、schemaを新旧binary互換で段階移行します。
08	deploy・logs・smoke・custom domain	deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。
09	scale・region・cost・Turso latency	scaleで共有されない状態を洗い出し、latency・capacity・costを一緒に測ります。
10	rollback・backup・disaster recovery・更新	rollback・復元・更新はbinary、schema、secret、asset、dataを一体で演習します。

完了条件

- ✓ image digestとcommitを対応付けた
- ✓ 全checkとpost-deploy smokeが通る
- ✓ migrationが旧imageと互換である
- ✓ isolated restore drillとrollback記録がある

REVIEW

巻末確認問題

用語だけでなく、「小さな例」「バックエンドでの場所」「失敗時の挙動」を含めて教えてください。

1. 01. release build・reproducibilityを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
2. 02. multi-stage Dockerfileを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
3. 03. Topcoat asset release bundleを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
4. 04. fly launch・fly.toml・portを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
5. 05. secrets・config・rotationを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
6. 06. health check・startup・readinessを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
7. 07. migration deploy順・release commandを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
8. 08. deploy・logs・smoke・custom domainを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
9. 09. scale・region・cost・Turso latencyを使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。
10. 10. rollback・backup・disaster recovery・更新を使う前と使った後で、値の流れと失敗の見え方はどう変わりますか。

答える順序 まず資料を閉じて説明し、次に最小コードを書き、最後に該当単元へ戻って不足を補います。正解の暗記ではなく、根拠を再現できることが目標です。

REVIEW ANSWERS

確認問題・解答の骨格

次の要点は模範解答の丸暗記用ではありません。自分の説明に「定義・小さな例・バックエンドでの場所」が含まれているかを照合します。

01. release build・reproducibility

release build・reproducibilityとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: source commit、lock、toolchain、artifact digestを一組で記録します。

02. multi-stage Dockerfile

multi-stage Dockerfileとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: runtime imageへbinary・assets・必要libraryだけを非rootで配置します。

03. Topcoat asset release bundle

Topcoat asset release bundleとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: binaryとasset bundleを同じbuildの不可分なartifactとして配布します。

04. fly launch・fly.toml・port

fly launch・fly.toml・portとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: listen addressとportをアプリ・container・Fly設定で一致させます。

05. secrets・config・rotation

secrets・config・rotationとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: 秘密は値ではなく名前・所有者・期限・rotation手順を管理します。

06. health check・startup・readiness

health check・startup・readinessとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: checkの目的をtraffic制御とprocess生存で分け、依存障害時の再起動loopを防ぎます。

07. migration deploy順・release command

migration deploy順・release commandとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: migration成功をreleaseの前提にし、schemaを新旧binary互換で段階移行します。

08. deploy・logs・smoke・custom domain

deploy・logs・smoke・custom domainとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。

09. scale・region・cost・Turso latency

scale・region・cost・Turso latencyとは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: scaleで共有されない状態を洗い出し、latency・capacity・costと一緒に測ります。

10. rollback・backup・disaster recovery・更新

rollback・backup・disaster recovery・更新とは何か、どの問題を防ぐか、バックエンドのどこで使うかを30秒で説明してください。

要点: rollback・復元・更新はbinary、schema、secret、asset、dataを一体で演習します。

GLOSSARY

重要用語と実務での位置

用語	一文定義	バックエンドでの位置
release build・reproducibility	source commit、lock、toolchain、artifact digestを一組で記録します。	deploy後にどのsourceと依存から作られたbinaryか追跡でき、同じcommitから再buildして比較できます。
multi-stage Dockerfile	runtime imageへbinary・assets・必要libraryだけを非rootで配置します。	0.5.0のbundle出力target/assetsをbinary隣の/app/assetsへコピーします。build stageのsourceやCargo registryをruntimeへ入れません。
Topcoat asset release bundle	binaryとasset bundleを同じbuildの不可分なartifactとして配布します。	cargo runだけでAssetBundle::loadを使うとbundleがなく起動失敗する場合があります。開発はtopcoat dev、本番はbuild→bundleです。
fly launch・fly.toml・port	listen addressとportをアプリ・container・Fly設定で一致させます。	127.0.0.1 bindやport不一致はdeploy成功でも到達不能になります。Docker local smokeで同じenvを先に試します。
secrets・config・rotation	秘密は値ではなく名前・所有者・期限・rotation手順を管理します。	secret更新はMachine再起動を伴います。新旧token共存→新token deploy・疎通→旧token revokeで切断時間を避けます。
health check・startup・readiness	checkの目的をtraffic制御とprocess生存で分け、依存障害時の再起動loopを防ぎます。	DB一時障害でMachine再起動loopにしないよう、何をcheckへ含めるかを決めます。ready失敗はtraffic除外、health失敗はprocess問題に寄せます。
migration deploy順・release command	migration成功をreleaseの前提にし、schemaを新旧binary互換で段階移行します。	SQLite/libSQLのALTER制約を確認し、大表のindex作成やbackfillを一度に行わず負荷とlockを測ります。
deploy・logs・smoke・custom domain	deploy完了は公開URLのHTTP・asset・auth・DB・logをsmokeして判断します。	smokeは専用test account/dataを使い、本番利用者データを変更しません。Set-Cookie、HTTPS redirect、security headersも確認します。
scale・region・cost・Turso latency	scaleで共有されない状態を洗い出し、latency・capacity・costを一緒に測ります。	複数MachineでもsessionをDBへ置くためsticky session不要です。in-memory rate limitやjob lockは分散状態として再設計します。
rollback・backup・disaster recovery・更新	rollback・復元・更新はbinary、schema、secret、asset、dataを一体で演習します。	Topcoatやlibsql更新は一度にまとめず、changelog、compile、test、bundle、migration、loadを確認し、rollback windowを残します。

PRIMARY SOURCES

公式資料と更新確認

仕様は更新されます。教材で意図を理解した後、実装時点の一次資料でAPI、security note、breaking changeを確認してください。

1. Rust Book

<https://doc.rust-lang.org/stable/book/>
Rust 2024 Edition を前提にした公式入門書

2. Rust Reference

<https://doc.rust-lang.org/reference/>
言語仕様を確認する一次資料

3. Standard Library

<https://doc.rust-lang.org/std/>
標準ライブラリAPI

4. Cargo Book

<https://doc.rust-lang.org/cargo/>
ビルド、依存関係、workspace、features

5. Rustdoc Book

<https://doc.rust-lang.org/rustdoc/>
APIドキュメントの書き方

6. Clippy

<https://doc.rust-lang.org/clippy/>
Rust公式Lint

7. Async Book

<https://rust-lang.github.io/async-book/>
非同期Rustの公式解説

8. Tokio Tutorial

<https://tokio.rs/tokio/tutorial>
Tokio公式チュートリアル

9. Topcoat

<https://github.com/tokio-rs/topcoat>
Topcoat公式リポジトリ。early-stageの注意を含む

10. Topcoat 0.5.0 API

<https://docs.rs/topcoat/0.5.0/topcoat/>
本教材で固定するAPI

11. Turso Rust SDK

<https://docs.turso.tech/sdk/rust/quickstart>
libSQL Rust SDK公式手順

12. Better Auth Database

<https://better-auth.com/docs/concepts/database>
Better AuthのDB構成

13. Fly.io Deploy

<https://fly.io/docs/launch/deploy/>
fly launch / fly deploy公式手順

14. Fly.io Health Checks

<https://fly.io/docs/reference/health-checks/>
公開後のヘルスチェック

15. OWASP Cheat Sheets

<https://cheatsheetseries.owasp.org/>
Webセキュリティの実務チェック

採用判断 本教材はRust 2024 Edition、Topcoat 0.5.0固定、Turso/libSQL、Topcoat Session + Argon2id、Fly.ioを主経路にします。Better AuthはTypeScript sidecarが適切な要件で比較します。